

Prakhar Shekhar Parthasarthi

Software Engineer · Generative AI · LLMs · AWS Bedrock · Java backends

prakhar.mnnit.2022@gmail.com

Shahjahanpur, UP · open to relocate

github.com/praxstack | linkedin.com/in/prakharshekhar | prax-portfolio-one.vercel.app

3

merged Fabric PRs

63

pipeline tests

v2.4.1

npm package

47%

p90 latency cut

SUMMARY

Software engineer shipping **generative AI and LLM integrations** on Java/AWS backends. Amazon Travel: owned **4 microservices** — p90 cut 47% (1.1s → 0.59s), conversion up +33%, webhook at 43 bps CPT. Three merged **Fabric Bedrock PRs** (42k stars), **Voxtral + Claude** pipeline (63 tests), and **npm v2.4.1**.

SKILLS

Generative AI & LLMs

Generative AI & LLMs: AWS Bedrock Converse/streaming, Claude repair, Voxtral ASR, structured AI markdown exports, multi-provider routing, bearer-token (ABSK) auth, BYOK proxies

Java & JVM

Java 17, Spring Boot, Spring MVC, Guice, JUnit 5, Mockito, SLF4J, Maven, Gradle · `ExecutorService`, `CompletableFuture`, `ConcurrentHashMap`, JVM profiling, GC analysis

AWS

Lambda (Java), DynamoDB, SQS, Step Functions, API Gateway, AppSync, CloudWatch, ECS, **CDK (Java)**

Distributed Systems

Microservices, REST, GraphQL, event-driven architecture, idempotency & dedup keys, state machines, retry/backoff, DLQ, at-least-once, TDD

Front-end

React.js (SSR & CSR), Redux, HTML/CSS, JavaScript/TypeScript, Mermaid, KaTeX, BYOK UI

Tooling & Process

Git, GitHub, CI/CD, Gradle JaCoCo coverage gates, Agile/Scrum, on-call, code review, incident response · Python, C++

EXPERIENCE

Software Development Engineer, Jul 2022 — Sep 2025

Amazon · Travel (Hotels & Flights) · Bengaluru

- Service ownership — Spring Boot, Spring MVC. Owned **4 customer-facing microservices** powering the hotel booking platform — search, listing, room selection, order review — across design, code review, deploy, and on-call. REST controllers, service-layer logic, `@Repository` DynamoDB DAOs via Spring DI. Prime-exclusive discount and partner-discount integration — ~30 bps funnel churn drop.
- JVM concurrency — `CompletableFuture`, bounded executors. Refactored sequential downstream calls into `CompletableFuture.allOf()` on a bounded `ExecutorService`; parallelised 6 partner-API lookups — hotel landing p90 cut 47% (1.1s → 0.59s), conversion up +33%. GZIP on 150–200 MB catalog payloads cut wire size 95%.
- Event-driven — SQS + Step Functions. Designed idempotent Java webhook for real-time flight schedule changes — SQS ingestion, DynamoDB dedup keys, audit logging, automated offline modifications — 43 bps CPT. Orchestrated booking state machine (confirm/cancel/retry) on Step Functions with Java Lambda handlers.
- Testing — JUnit 5, Mockito, Spring Boot. Held 80%+ line coverage; Mockito argument captors for downstream contract verification; `@SpringBootTest` integration tests against in-memory DynamoDB. Introduced Gradle JaCoCo coverage gates in CI.

EXPERIENCE · AMAZON SDE (CONTINUED)

Jul 2022 — Sep 2025 · Travel (Hotels & Flights) · Bengaluru

- Infrastructure as code — AWS CDK (Java). Authored all service IaC in **Java CDK** — Lambdas, DynamoDB, SQS, Step Functions, API Gateway, CloudWatch alarms. Migrated 2 services from hand-rolled CloudFormation — rollback incidents halved.
- Operational excellence. On-call tickets cut from 20+ to ~5 per shift (~2 SDE-weeks/week reclaimed). Java email-automation deprecated stale WebLab experiments — config-hygiene incidents from 5–6 to 0. Resolved 150+ production incidents; authored COEs. Built auto-insurance purchase flow (Java + React) — 60%+ renewal lift.

Software Development Engineer, Intern, May 2021 – Jul 2021 · Feb 2022 – Jul 2022

Amazon · Bengaluru

- Migrated legacy GraphQL resolvers to **Java** — DynamoDB via Spring-injected DAOs, API Gateway + Lambda.
- Shipped **One-Click Renewal** for auto-insurance using past-purchase data — async Java pre-fill workflow.
- Applied OO design and **Google Guice DI**; 90%+ unit-test coverage (JUnit 5 + Mockito).

PROJECTS

Audio Transcription Pipeline

github.com/praxstack/audio-transcription-pipeline

Python Bedrock pyannote pytest

Multi-stage pipeline: raw audio/video → speaker-labelled Markdown. pyannote diarization + AWS Bedrock (Voxtral ASR + Claude repair).

63 tests · 6 stages · MIT.

Warp BYOK Proxy

github.com/praxstack/warp-byok-proxy · AGPL-3.0

Rust AWS Bedrock BYOK

Local proxy that routes Warp Terminal AI calls to an operator-controlled AWS Bedrock account. Bearer-token and AWS credential auth; protobuf chat mapped onto Bedrock Converse streaming.

113 Rust tests.

AI Visual Code Review

github.com/praxstack/ai-visual-code-review · npm

Node.js TypeScript Jest VS Code API

Security-hardened visual code-review tool with GitHub-style diff UI, AI markdown exports, command-injection protection, rate limiting. Published npm + VS Code extension.

Published: npm v2.4.1 · MIT.

Redis Server in Java

github.com/praxstack/redis-server-java

Java 17 ConcurrentHashMap JUnit 5

RESP2-compatible Redis from scratch; bounded `ExecutorService`, lock-free reads, hybrid TTL eviction, TCP integration tests.

GET 195k rps · SET 154k rps · p50 < 0.3 ms · 42 tests.

Open-Source Contributions

Merged into danielmiessler/Fabric (Go, 42k stars):

#2044 — AWS Bedrock bearer-token (ABSK) auth + guided `--setup`. +882 / -83

#2061 — streaming-goroutine deadlock fix in `Chatter.Send`. +494 / -42

#2052 — dynamic Bedrock region fetching (40+ regions). +252 / -15

Also merged: deer-flow (model-config normalization).

EDUCATION

Master of Computer Applications (M.C.A.), Computer Science · MNNIT, Prayagraj · GPA 9.25 · 2019 — 2022

Bachelor of Science (B.Sc. Hons), Computer Science · Bhaskaracharya College, University of Delhi · 2015 — 2018

AWARDS

Amazon Pay Merchants Categories Champion — 100+ hotel trust-buster bugs; on-call 20+ to ~5 per shift · 2023